

Sprawozdanie z Projektu Podstawy Podejmowania Decyzji

Mieszko Czubiński, nr albumu: 284047

12 czerwca 2026

1 Wstęp

1.1 Opis

Badany problem jest trójwymiarowym problemem pakowania do kontenerów (ang. *bin packing problem*). Dodatkowo wprowadzono zależności między pakowanymi przedmiotami oraz uwzględniono ciągłe dostarczanie kolejnych produktów do rozpatrzenia. Celem jest minimalizacja liczby kontenerów potrzebnych do przewiezienia przedmiotów.

Rozwiązanie tego problemu może mieć zastosowanie przy załadunku towarów z fabryki do ciężarówek dostawczych. Dane użyte w przykładzie nawiązują do fabryki mebli. Produktem końcowym takiej fabryki są części mebli zapakowane w prostopadłościenną kartony. Ponieważ jeden mebel może być rozłożony na więcej niż jeden karton, wprowadzono między nimi zależności.

Fabryka nieustannie produkuje kolejne meble, dlatego istotne jest ciągłe odbieranie towaru przez ciężarówki. Są one stopniowo doładowywane napływającymi kartonami, gdy któraś jest dostatecznie zapełniona, zostaje wysłana w trasę i nie podlega już żadnym zmianom.

1.2 Obiekt podejmowania decyzji

Rozpatrywany system analizowany jest z perspektywy systemowej jako obiekt przetwarzający *decyzje* na *ocenę* tych decyzji.

Decydentem jest algorytm optymalizacyjny odpowiedzialny za załadunek w fabryce mebli, natomiast *przedmiotem decyzji* jest przydział kartonów z ciągle napływającego strumienia produkcji do kolejnych ciężarówek.

Wejściem (*decyzją*) jest przypisanie każdego kartonu do konkretnego kursu ciężarówki. *Zbiór decyzji dopuszczalnych* tworzą te przydziały, które jednocześnie:

- mieszczą się geometrycznie w wymiarach naczepy i nie kolidują ze sobą,
- nie przekraczają dopuszczalnej ładowności (limitu wagi),
- spełniają zależności między kartonami – karton może zostać załadowany tylko wtedy, gdy w tym samym kursie znajdują się wszystkie kartony, od których zależy.

Wyjściem (*oceną decyzji*) jest całkowita liczba ciężarówek użytych do przewiezienia całości towaru.

Zależność wyjścia od wejścia – sposób rozmieszczenia kartonów oraz reguły zależności określają, ile kartonów zmieści się w jednym kursie, a przez to – ile kursów potrzeba na obsłużenie całego strumienia produkcji. Kartony przydzielane są na bieżąco. Wysłanej ciężarówki nie można już zmienić.

1.3 Wymagania systemowe

Wymaganie na wyjście – kryterium decyzyjnym jest minimalizacja liczby użytych ciężarówek. Ze względu na NP-trudność problemu nie dąży się do gwarantowanego optimum, lecz do rozwiązania możliwie bliskiego optymalnemu.

Typ systemu – jest to *system automatycznego podejmowania decyzji*, w którym algorytm samodzielnie wyznacza plan załadunku przydzielanych kartonów, bez udziału człowieka.

2 Problem optymalizacyjny

Minimalizowana jest liczba ciężarówek (kursów) potrzebnych do przewiezienia wszystkich kartonów, przy zachowaniu ograniczeń przestrzennych, wagowych i zależnościowych.

2.1 Oznaczenia

- $I = \{1, \dots, n\}$ – zbiór kartonów; karton i : wymiary (w_i, h_i, d_i) , waga m_i ,
- $K = \{1, \dots, K_{\max}\}$ – zbiór dostępnych ciężarówek; naczepa: wymiary (W, H, D) , ładowność Q ,
- $\mathcal{D} \subseteq I \times I$ – zbiór zależności: $(i, j) \in \mathcal{D}$ oznacza, że karton i wymaga kartonu j w tym samym kursie.

2.2 Zmienne decyzyjne

$$y_k = \begin{cases} 1 & \text{ciężarówka } k \text{ jest użyta} \\ 0 & \text{w przeciwnym razie,} \end{cases} \quad x_{ik} = \begin{cases} 1 & \text{karton } i \text{ jedzie ciężarówką } k \\ 0 & \text{w przeciwnym razie,} \end{cases}$$

2.3 Funkcja celu

$$\min \sum_{k \in K} y_k$$

2.4 Ograniczenia

$$\sum_{k \in K} x_{ik} = 1, \quad \forall i \in I \quad (1)$$

$$x_{ik} \leq y_k, \quad \forall i \in I, k \in K \quad (2)$$

$$\sum_{i \in I} m_i x_{ik} \leq Q y_k, \quad \forall k \in K \quad (3)$$

$$x_{ik} \leq x_{jk}, \quad \forall (i, j) \in \mathcal{D}, k \in K \quad (4)$$

Ograniczenia (1) zapewniają, że każdy karton zostanie przewieziony dokładnie raz, (2) – kartony trafiają wyłącznie do użytych ciężarówek, (3) – nieprzekroczenie ładowności, a (4) – spełnienie zależności między kartonami.

Dodatkowo wymagany jest warunek nienakładania się kartonów przewożonych tą samą ciężarówką: dla każdej pary $i < j$ takiej, że $x_{ik} = x_{jk} = 1$, co najmniej jeden z prostopadłościanów leży w całości po jednej stronie drugiego wzdłuż którejś z osi.

3 Rozwiązanie

Trójwymiarowy problem pakowania z minimalizacją liczby kontenerów jest NP-trudny razem z dodatkowymi ograniczeniami (zależności, ciągły napływ kartonów oraz wysyłka aut w trybie online) wyklucza rozwiązanie dokładne. W celu rozwiązania problemu zastosowano metaheurystykę – *algorytm genetyczny*.

3.1 Reprezentacja permutacyjna i dekodery

Algorytm nie decyduje wprost, który karton trafi do której ciężarówki. Ustala jedynie kolejność π , w jakiej kartony są rozpatrywane przy załadunku – i to tę kolejność optymalizuje. Dla zadanej kolejności prosta procedura (*dekoder*) ładuje kartony jeden po drugim do ciężarówek, dając gotowy rozkład kartonów po autach:

- kartony rozpatrywane są w kolejności zadanej przez π ,
- kartony powiązane zależnościami tworzą *grupę*, która musi jechać tym samym autem; grupy wyznacza funkcja `build_groups`, łącząc ze sobą kartony zależne i pomijając te już wysłane; grupę ładuje się w całości albo wcale,
- każda grupa trafia do pierwszej ciężarówki, w której się mieści; jeśli nie mieści się w żadnej z otwartych – otwierana jest nowa,

Ułożenie kartonów wewnątrz auta wyznacza heurystyka *punktów ekstremalnych*. Oparta na zbiorze wolnych narożników, w które można wstawić karton. Karton trafia do najniższego i najgłębszego narożnika, w którym się mieści i nie nachodzi na inne kartony, po wstawieniu wzdłuż jego krawędzi powstają nowe naroża.

Zaletą oddzielenia kolejności od dekodera jest to, że algorytm nigdy nie tworzy załadunku łamiącego ograniczenia – przeszukuje tylko różne kolejności, a każda z nich daje poprawny plan.

```

def pack(order, gene_ids, by_id, groups, truck, departed=()):
    assigned = set(departed)
    bins = []
    for g in order:
        iid = gene_ids[int(g)]
        if iid in assigned:
            continue
        group = [by_id[c] for c in groups[iid]]
        if not any(b.add_group(group) for b in bins):
            new_bin = Bin(truck)
            new_bin.add_group(group)
            bins.append(new_bin)
        assigned.update(groups[iid])
    return bins

```

3.2 Funkcja przystosowania

Gdyby oceniać rozwiązanie wprost liczbą aut, wiele różnych kolejności miałoby identyczną ocenę i algorytm nie wiedziałby, które z nich są bliżej usunięcia kolejnego auta. Dlatego algorytm maksymalizuje ocenę, która nagradza auta wypełnione jak najpełniej:

$$F(\pi) = \frac{1}{m} \sum_{k=1}^m \left(\frac{V_k}{V} \right)^2,$$

gdzie m – liczba użytych aut, V_k – objętość zajęta w aucie k , a $V = W \cdot H \cdot D$ – objętość ładowni. Podniesienie do kwadratu nagradza rozkłady, w których jedno auto są pełne, a jedno prawie puste – takie najłatwiej domknąć, wyrzucając ostatnie, słabo zapełnione auto.

```

def fitness(order, items, truck, departed):
    groups = packer.build_groups(items, departed)
    gene_ids = [it.id for it in items]
    by_id = {it.id: it for it in items}
    bins = packer.pack(order, gene_ids, by_id, groups, truck,
                        departed)
    capacity = truck.volume
    return sum((b.used_volume / capacity) ** 2 for b in bins) /
           len(bins)

```

3.3 Operatory genetyczne

- **Selekcja turniejowa** – losuje się t kolejności, a dalej przechodzi najlepsza z nich;
- **Mutacja przez zamianę** – w kolejności z prawdopodobieństwem p_m zamienia się miejscami dwa kartony;
- **Elityzm** – kilka najlepszych kolejności przechodzi do następnego pokolenia bez zmian.

3.4 Obsługa ciągłych dostaw

Co ustaloną liczbę pokoleń `ARRIVAL_INTERVAL` do każdej kolejności dopisywane są nowo przybyłe kartony, a ocena liczona jest już dla powiększonego zbioru. Obliczona wcześniej kolejność zostaje zachowana.

3.5 Odjazdy ciężarówek

Aby oddać cykl załadunku, ciężarówki wysyłane są w trakcie pracy algorytmu. Co `DEPART_INTERVAL` pokoleń najpełniejsze z otwartych aut zostaje wysłane: jego kartony trafiają do zbioru `departed` i nie biorą udziału w dalszych obliczeniach (dekoder je pomija). Żeby żaden karton nie został zapakowany dwa razy, wysłane kartony (`departed`) pomijane są i przy ładowaniu (`pack`), i przy tworzeniu grup zależności (`build_groups`).

```
if gen % config.DEPART_INTERVAL == 0 and bins:
    fullest = max(bins, key=lambda b: b.used_volume)
    shipped.append(fullest)
    departed_ids.update(fullest.contents)
    print(f"Truck departed: {len(fullest.contents)} boxes, {100*fullest.used_volume/truck.volume:.1f}% full")
```

Końcowy wynik to liczba aut wysłanych w trakcie symulacji powiększona o auta, które na końcu wciąż są doładowywane.

3.6 Parametry i instancja testowa

Jako ciężarówkę przyjęto realny lekki samochód dostawczy klasy o parametrach ładowni: $300 \times 180 \times 200$ cm (dł. $\times$ szer. $\times$ wys.) i ładowności 1500 kg. Symulacja startuje z 10 kartonami i co 10 pokoleń dostarcza po 5 nowych, co przy 200 pokoleniach daje 105 kartonów do rozwiezenia. Równolegle co 40 pokoleń wysyłana jest najpełniejsza ciężarówka.

Parametr	Wartość
Rozmiar populacji	50
Liczba pokoleń	200
Przedmioty początkowe	10
Interwał dostaw [pokolenia]	10
Liczność dostawy	5
Interwał odjazdów [pokolenia]	40
Ziaro generatora	42

Tabela 1: Parametry algorytmu genetycznego i symulacji dostaw.

3.7 Model dokładny w CPLEX

Ten sam problem zapisano dodatkowo w programie IBM ILOG CPLEX (plik `binpacking3d.mod` – model, `binpacking3d.dat` – dane). CPLEX to tzw. *solver*: sam sprawdza możliwe załadunki i zwraca najlepszy, z gwarancją, że lepszego nie ma – czego algorytm genetyczny nie zapewnia. Dlatego wynik CPLEX-a służy jako *wzorzec* jakości.

Ceną za tę pewność jest konieczność opisanie wzorami wszystkiego – także ułożenia kartonów, które dekodery załatwiał sam. W modelu dochodzą więc zmienne mówiące, jak karton jest obrócony, gdzie stoi i jak kartony są rozsunięte, by na siebie nie nachodziły.

Celem jest liczba użytych aut, którą minimalizujemy. Dochodzą do tego proste warunki: każdy karton jedzie dokładnie jednym autem, ma dokładnie jedno ustawienie, a suma wag w aucie nie przekracza ładowności:

```
minimize sum(k in K) y[k];

subject to {
    forall(i in I)
        sum(k in K) x[i][k] == 1; // karton raz, jednym autem
    forall(i in I)
        sum(o in O) ori[i][o] == 1; // dokładnie jedno ustawienie
    forall(k in K)
        sum(i in I) weight[i]*x[i][k] <= Q*y[k]; // limit wagi w aucie
}
```

Karton można postawić na 6 sposobów. Wybrane ustawienie decyduje, ile zajmuje wzdłuż każdej osi naczepy – w kodzie liczą to lx, ly, lz. Wymagamy tylko, by karton nie wystawał poza naczepę:

```
dexpr float lx[i in I] = sum(o in O) boxDim[i][perm[o][1]] *
    ori[i][o];
dexpr float ly[i in I] = sum(o in O) boxDim[i][perm[o][2]] *
    ori[i][o];
dexpr float lz[i in I] = sum(o in O) boxDim[i][perm[o][3]] *
    ori[i][o];

forall(i in I) {
    px[i] + lx[i] <= W; // mieści się wzdłuż szerokości
    py[i] + ly[i] <= H; // ... wysokości
    pz[i] + lz[i] <= D; // ... głębokości
}
```

Dwa pudełka się nie nakładają, gdy z którejś strony jedno stoi w całości obok drugiego (na lewo, prawo, niżej, wyżej, z przodu lub z tyłu – 6 możliwości). Dla każdej pary dodajemy 6 „przełączników” sep, a technika *big-M* sprawia, że włączony przełącznik realnie rozsuwa kartony, a wyłączony czyni warunek nieistotnym. Ostatnia linia pilnuje, by rozsuwać tylko kartony jadące tym samym autem:

```
forall(p in Pairs) {
    sepXleft:   px[p.i] + lx[p.i] <= px[p.j] + W * (1 - sep[p][1]);
    sepXright:  px[p.j] + lx[p.j] <= px[p.i] + W * (1 - sep[p][2]);
    sepYbelow:  py[p.i] + ly[p.i] <= py[p.j] + H * (1 - sep[p][3]);
    sepYabove:  py[p.j] + ly[p.j] <= py[p.i] + H * (1 - sep[p][4]);
    sepZback:   pz[p.i] + lz[p.i] <= pz[p.j] + D * (1 - sep[p][5]);
    sepZfront:  pz[p.j] + lz[p.j] <= pz[p.i] + D * (1 - sep[p][6]);

    // rozdzielanie wymuszane tylko gdy oba w tej samej ciężarówce
    forall(k in K)
        noOverlap: sum(t in S) sep[p][t] >= x[p.i][k] + x[p.j][k]
            - 1;
}
```

Przełączników **sep** jest 6 na każdą parę kartonów, więc ich liczba rośnie z kwadratem liczby kartonów – model szybko puchnie i CPLEX liczy coraz dłużej. Udowodnione optimum osiąga się tylko dla kilkunastu kartonów; dlatego pełny strumień produkcji obsługuje algorytm genetyczny, a CPLEX sprawdza jego jakość na małej próbce.

4 Wyniki

Wyniki przedstawiono w dwóch częściach. Najpierw, na małej instancji 15 kartonów (z zależnościami $1 \rightarrow 2$, $3 \rightarrow 4$, $6 \rightarrow 7$ oraz $7 \rightarrow 8$), porównano algorytm genetyczny z modelem dokładnym – rozmiar dobrano tak, by CPLEX zdążył wyznaczyć gwarantowane optimum stanowiące punkt odniesienia. Następnie uruchomiono pełną symulację ciągłego strumienia produkcji, dla której metoda dokładna jest już poza zasięgiem.

4.1 Model dokładny (CPLEX)

CPLEX wyznaczył rozwiązanie **optymalne** o wartości funkcji celu równej **1** – wszystkie 15 kartonów zmieściło się w jednej ciężarówce, przy spełnieniu ograniczeń wagi, zależności i nienakładania. Wynikowe rozmieszczenie (położenie naroża oraz wymiary kartonu po obrocie):

#	Karton	Położenie (x, y, z)	Wymiary $(w \times h \times d)$
1	Dining Table Top	(0, 0, 2)	$10 \times 180 \times 90$
2	Table Legs Set	(0, 90, 297)	$15 \times 80 \times 15$
3	Wardrobe Body	(45, 93, 112)	$60 \times 100 \times 200$
4	Wardrobe Doors	(175, 0, 112)	$10 \times 60 \times 200$
5	Sofa 3-Seater	(0, 0, 92)	$100 \times 90 \times 220$
6	Queen Mattress	(105, 0, 112)	$30 \times 160 \times 200$
7	Bed Frame	(135, 0, 102)	$40 \times 160 \times 210$
8	Headboard	(175, 60, 110)	$10 \times 120 \times 160$
9	Bookshelf	(15, 90, 110)	$30 \times 80 \times 180$
10	Office Chair	(75, 90, 0)	$60 \times 60 \times 110$
11	Drawer Unit	(135, 0, 52)	$50 \times 70 \times 50$
12	Nightstand	(35, 45, 0)	$40 \times 45 \times 55$
13	Coffee Table	(30, 90, 0)	$45 \times 60 \times 110$
14	TV Stand	(15, 0, 0)	$150 \times 45 \times 50$
15	Dresser	(135, 73, 0)	$50 \times 120 \times 90$

Tabela 2: Rozmieszczenie 15 kartonów w rozwiązaniu optymalnym (CPLEX).

4.2 Algorytm genetyczny (Python)

Algorytm genetyczny, choć nie korzysta z żadnej wiedzy o optimum, również upakował wszystkie 15 kartonów w **jednej** ciężarówce:

#	Karton	Położenie (x, y, z)	Wymiary $(w \times h \times d)$
1	Dining Table Top	(170, 0, 0)	$10 \times 90 \times 180$
2	Table Legs Set	(160, 120, 0)	$15 \times 80 \times 15$
3	Wardrobe Body	(0, 0, 0)	$60 \times 100 \times 200$
4	Wardrobe Doors	(60, 0, 0)	$10 \times 60 \times 200$
5	Sofa 3-Seater	(70, 0, 0)	$90 \times 100 \times 220$
6	Queen Mattress	(0, 140, 0)	$160 \times 30 \times 200$
7	Bed Frame	(0, 100, 0)	$160 \times 40 \times 210$
8	Headboard	(160, 0, 0)	$10 \times 120 \times 160$
9	Bookshelf	(0, 170, 0)	$80 \times 30 \times 180$
10	Office Chair	(0, 0, 240)	$60 \times 110 \times 60$
11	Drawer Unit	(110, 140, 250)	$70 \times 50 \times 50$
12	Nightstand	(0, 0, 200)	$45 \times 55 \times 40$
13	Coffee Table	(0, 140, 250)	$110 \times 60 \times 45$
14	TV Stand	(0, 140, 200)	$150 \times 45 \times 50$
15	Dresser	(70, 0, 220)	$90 \times 120 \times 50$

Tabela 3: Rozmieszczenie 15 kartonów wyznaczone przez algorytm genetyczny.

4.3 Porównanie

Metoda	Ciężarówka	Wypełnienie obj.	Wykorzyst. ładowności	Optymaln
Model dokładny (CPLEX)	1	76,4%	37,7%	optimum (z do
Algorytm genetyczny	1	76,4%	37,7%	zgodne z opt

Tabela 4: Porównanie obu metod na tej samej instancji 15 kartonów (naczepa $180 \times 200 \times 300$, ładowność 1500).

4.4 Pełna symulacja strumienia produkcji

Właściwy scenariusz – ciągły napływ kartonów i stopniowa wysyłka aut – uruchomiono na pełnych danych. Symulacja startuje z 10 kartonami i co 10 pokoleń otrzymuje 5 nowych (łącznie 105 kartonów w 200 pokoleniach), a co 40 pokoleń wysyłana jest najpełniejsza ciężarówka. W ciągu 200 pokoleń wyjechało 5 aut, a szóste pozostało w załadunku – razem **6 ciężarówek** na 105 kartonów.

Każdy napływ chwilowo pogarsza ocenę (nowe kartony otwierają dodatkowe auto), po czym algorytm ją odrabia, dopakowując je do aut już otwartych.

Cieżarówka	Kartony	Wypełnienie obj.	Wykorzyst. ładowności	Status
1	18	88,2%	44,3%	wysłana (pok. 40)
2	19	80,0%	55,4%	wysłana (pok. 80)
3	19	81,1%	68,5%	wysłana (pok. 120)
4	17	78,0%	67,1%	wysłana (pok. 160)
5	18	79,8%	65,3%	wysłana (pok. 200)
6	14	67,6%	67,9%	w załadunku

Tabela 5: Wynik pełnej symulacji: 105 kartonów rozwiezionych 6 ciężarówkami (5 wysłanych w trakcie pracy, 1 wciąż doładowywana).

5 Wnioski

W ramach projektu zbudowano automatyczny system załadunku, rozwiązujący trójwymiarowy problem pakowania z dodatkowymi ograniczeniami – zależnościami między kartonami oraz ciągłym napływem produkcji i wysyłką aut. Sercem rozwiązania jest algorytm genetyczny operujący na kolejności kartonów, z dekoderm opartym na heurystyce punktów ekstremalnych; dzięki temu każde rozważane rozwiązanie jest z założenia poprawne, a algorytm przeszukuje wyłącznie sensowne plany załadunku.

Jakość metaheurystyki sprawdzono, porównując ją z modelem dokładnym (CPLEX) na tej samej, małej instancji 15 kartonów. Obie metody użyły dokładnie jednej ciężarówki, zatem **algorytm genetyczny osiągnął wynik optymalny** – tyle samo aut, ile gwarantuje solver. Rozmieszczenia kartonów się różnią (CPLEX swobodnie dobiera współrzędne, dekodery wkłada kartony w kolejne wolne naroża), lecz dla funkcji celu, czyli liczby aut, nie ma to znaczenia.

Trzeba jednak zaznaczyć, że była to instancja łatwa – cały zestaw mieścił się w jednym aucie, więc nie różnicuje obu metod. Przy większej liczbie kartonów lub cieńszej naczepie algorytm genetyczny mógłby użyć o jedno auto więcej niż optimum, ale takich rozmiarów model dokładny nie rozwiązuje już w rozsądnym czasie. To właśnie ograniczenie było głównym powodem sięgnięcia po metaheurystykę: w odróżnieniu od solvera radzi sobie ona z pełnym, wciąż rosnącym strumieniem produkcji, dla którego dokładne optimum pozostaje poza zasięgiem. W przeprowadzonej symulacji algorytm rozwiązał 105 kartonów sześcioma ciężarówkami, utrzymując wypełnienie wysyłanych aut na poziomie 78–88% objętości.